

Supplementary Material for: Semantic Understanding of Stored Household Items

Michaela Levi Richter¹, Reuth Mirsky², and Oren Glickman¹

¹ Department of Computer Science, Bar-Ilan University

² Department of Computer Science, Tufts University

1 Experimental Setup Details

1.1 Computing Environment

Most experiments were conducted on an Intel Core i7 machine with 16GB RAM. GPU-dependent models were run on a university cluster with four NVIDIA GTX 1080 Ti GPUs (11GB each), 125GB RAM, and a 48-core Intel Xeon CPU.

1.2 Baseline Implementations

Random For each item-image pair, we took the list of containers provided by Grounding-DINO during the annotation phase and picked one of them at random.

Grounding-DINO and SAM We used Grounding-DINO [2] with SAM [1] to detect and segment storage containers such as drawers and cabinet doors. For this pipeline, we ran the Grounding-DINO detection model with a textual prompt of the form:

```
drawer for {item} . cabinet door for {item}
```

The prompt follows recommendations from the Grounding-DINO authors: it is short, uses simple phrases separated by periods, and avoids complex syntax. The use of “for” (e.g., `drawer for {item}`) reflects a concise and structured phrasing style that specifies the relation between an item and a potential storage container.

The following thresholds were applied during inference: `box_threshold = 0.30` and `text_threshold = 0.25`, which control the minimum confidence required for detecting bounding boxes and associated text regions, respectively. From the resulting detections, we selected the bounding box with the **highest confidence** and passed it to the Segment Anything Model (SAM) to generate segmentation masks.

We used the official implementations of both models: `IDEA-Research/grounding-dino` and `facebook/sam-vit-h`. Grounding-DINO was used at commit `c023468` of the

IDEA-Research/grounding-dino repository. The SAM implementation corresponds to the official April 2023 release from Meta³, using the `vit-h` variant.

Grounding-DINO handled object-level bounding box detection conditioned on the textual prompt, while SAM produced binary segmentation masks for those regions. The segmentation was implemented using the SAM PyTorch interface, which transforms bounding boxes to input-specific image coordinates and outputs a binary mask for each box.

The segmentation masks were post-processed using OpenCV to extract polygon contours. We applied the Ramer-Douglas-Peucker (RDP) algorithm with $\text{epsilon} = 0.02 * \text{arcLength}$ to simplify the polygons.

It is important to note that this use of Grounding-DINO and SAM is distinct from the initial use of Grounding-DINO during dataset annotation, where all visible containers (drawers and cabinets) were detected in bulk. In the current evaluation phase, we specifically query for each item separately, prompting the model to localize where the item is likely stored. To ensure a fair evaluation without introducing annotation bias, a prediction was only considered correct if it achieved an $\text{IoU} \geq 0.95$ to allow only small correction with the ground truth container.

Grounding-DINO and SAM without Item Prompt To assess the influence of including the item name, we also ran Grounding-DINO with a minimal prompt - identical to the annotation phase - omitting the item entirely:

`drawer . cabinet door`

Again, we selected the container with the highest confidence score. Comparing these results to those obtained with the item-inclusive prompt allows us to quantify how much the item name boosts the model’s performance.

Kosmos-2 We used the Kosmos-2 MLLM [3] with both image and text input. The prompt followed a structured grounding format using the special tokens provided by the model:

`<grounding> In which<phrase> drawer</phrase> or<phrase> cabinet door</phrase> is<phrase> a {item}</phrase> stored?`

The prompt structure follows Kosmos-2’s grounding format, where the `<grounding>` tag activates grounding mode and each `<phrase>` tag marks a semantically important span. By marking both the container types (e.g., `drawer`, `cabinet door`) and the queried item, the model is guided to focus on relevant visual concepts and return grounded predictions accordingly.

The model’s outputs captions and entity bounding boxes relevant to the query. Since Kosmos-2 does not use a pre-defined list of container locations (unlike human annotators), we considered a prediction correct if the IoU between

³ <https://github.com/facebookresearch/segment-anything>

the model’s output and the ground truth was ≥ 0.5 (meaning that the bounding box overlapped the true container in at least 20% of its area).

We used the official HuggingFace implementation⁴. For inference, we used the model’s `generate()` method with the following settings:

- `use_cache=True`: enables reuse of key and value tensors from previous self-attention layers for efficient autoregressive decoding.
- `max_new_tokens=128`: limits the number of newly generated tokens to 128.
- `skip_special_tokens=True`: ensures that decoding omits any special tokens in the output.

After generation, we applied the `post_process_generation()` method twice: first with `cleanup_and_extract=False` to inspect raw model output, and again with default settings to extract grounded entity descriptions and their associated bounding boxes.

All inference was conducted on a single NVIDIA GPU, using PyTorch. We used the pre-trained checkpoint without any additional fine-tuning or modification.

Gemini (Google MLLM) We used Google’s Gemini API to evaluate multimodal performance. Specifically, we tested two model versions: `gemini-1.5-flash-latest` and `gemini-2.5-flash`. Predictions were considered correct if the IoU with ground truth was ≥ 0.5 , as in the Kosmos-2 evaluation.

Each image-item pair was processed using the following template:

```
Analyze the provided image of a kitchen.
Identify the item: '{item}'
Determine the most likely storage location for this item, considering
only drawers or cabinet doors visible in the image.
Provide the bounding box coordinates for this storage loc-
ation as a Python list of four integers: [x_min, y_min, x_max,
y_max].
The coordinates should be relative to the image dimensions (top-left
is [0,0]).
Only output the list of coordinates and nothing else. For example:
[100, 200, 300, 400]
If you cannot determine a likely storage location (drawer or
cabinet door) for this item, output: [0, 0, 0, 0]
```

The API was accessed via an official Google API key, with reliability safeguards including up to 3 retry attempts (starting with a 5-second delay), and a 1-second pause between successful calls. We saved progress every 50 processed examples.

⁴ <https://huggingface.co/microsoft/kosmos-2-patch14-224>

ChatGPT-4o We used the GPT-4o model with vision capabilities through the OpenAI API. Predictions were considered correct if the IoU with the ground truth polygon was ≥ 0.5 , consistent with other models.

The prompt format was:

```
You are analyzing a kitchen image.
The visible containers are drawers and cabinet doors.
The item to store is: {item}
Determine the most likely storage location among visible drawers
and cabinet doors only.
Return a list of 4-point bounding box coordinates for the item.
If a suitable location cannot be determined, return an empty
list [].
Only return the bounding box list, nothing else.
```

The API was accessed via an OpenAI API key. For each image-item pair, the image was encoded in base64 and sent alongside the prompt in a multimodal request. Responses were parsed to extract the bounding box list. Robustness measures included handling exceptions per request and adding a 1-second delay between API calls to avoid rate limits.

LLaMA-4 and Qwen-2.5 (Together AI API) We evaluated the multimodal LLaMA-4 [5] and Qwen-2.5 [4] models via the Together AI API, using the checkpoints `meta-llama/LLaMA-4-Maverick-17B-128E-Instruct-FP8` and `Qwen/Qwen2.5-VL-72B-Instruct`, respectively. As with the other models, predictions were considered correct if the IoU with the ground-truth container polygon was ≥ 0.5 .

The prompt format was identical to our GPT-4o setup:

```
You are analyzing a kitchen image.
The visible containers are drawers and cabinet doors.
The item to store is: {item}
Determine the most likely storage location among visible drawers
and cabinet doors only.
Return a list of 4-point bounding box coordinates for the item.
If a suitable location cannot be determined, return an empty
list [].
Only return the bounding box list, nothing else.
```

Each request included the Base64-encoded image and the text prompt in a single multimodal payload. We accessed the API using a Together AI API key, parsed the JSON response to extract the bounding box list, and converted it to our evaluation format.

1.3 Determining IoU threshold

We selected an IoU threshold of 0.5 based on a quality-weighted analysis of human-labeled predictions. Correct or partially correct predictions averaged an

IoU of 0.54, while incorrect predictions consistently fell below this range. For Grounding-DINO, we also report $\text{IoU} \geq 0.95$ to account for bounding-box imprecision and annotation bias.

To select an appropriate IoU threshold for our results, we first divided the models and baselines into two groups:

- **Models with prior container knowledge:** This includes human annotators (who chose from a predefined list of containers in the web application), the random baseline (which selects a random container from Grounding-DINO detections), Grounding-DINO with the item in the prompt (used both for object detection and for item storage), and NOAM (which also receives the list of detected containers).
- **Models without prior container knowledge:** This includes Gemini, GPT-4o, Kosmos-2, LLaMA-4, and Qwen-2.5.

For fair comparison, we required $\text{IoU}=1.0$ for the first group. For the second group, we examined cases with $\text{IoU} \geq 0.0$ and $\text{IoU} \geq 0.1$, performing a sanity check by reviewing all images, marking the “correct” container polygons, and assigning scores of 0 (incorrect), 0.5 (partially correct), or 1 (fully correct). Computing a quality-weighted average IoU from these scores yielded 0.54, which guided our choice of 0.5 as the threshold.

As shown in Figure 1, incorrect predictions (score = 0) never exceed $\text{IoU}=0.17$, while fully correct ones (score = 1) never fall below $\text{IoU} \approx 0.365$. Partially correct cases lie between $\text{IoU} \approx 0.088$ and 0.392. This gap suggests an ideal cutoff in $[0.17, 0.365]$. We adopt $\text{IoU} \geq 0.5$, which is stricter than the minimum correct bound, to maximize precision based on our quality-weighted analysis.

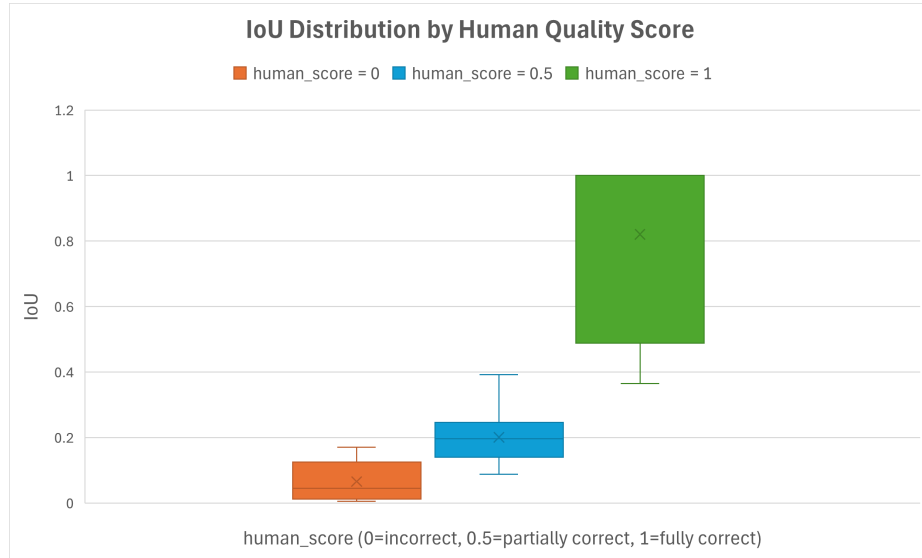


Fig. 1. IoU distributions across human-assigned quality scores (0=incorrect, 0.5=partially correct, 1=fully correct).

2 NOAM: Prompt Design and Task Framing

After generating descriptions for all containers in an image, we focused on designing an effective prompt to convey the task to the language model. All prompt variants were tested using ChatGPT-4, and the final selected prompt was also applied to LLaMA-3.3.

In every prompt format and for both models, the goal was to select the most likely container ID from the provided list, indicating where the queried item would typically be stored. If more than one container was retrieved (in case that two containers were selected as the most likely), only the first was used in the evaluation. If the model determined that none of the containers were suitable, it returned “None”.

Below are the main prompt strategies we explored:

1. Instructional Prompt: Service Robot Role Our first approach described the model as a service robot trying to locate an item in a kitchen image, based on a list of container descriptions:

```
prompt = f"""You are a service robot in a domestic environment. You are
now looking at a kitchen scene.
```

```
The containers (drawers, cabinet doors etc.) detected in the image are:
{chr(10).join(f"- {desc}" for desc in containers)}
```

```

For each of the following items:
{'', '.join(items)}

Please determine in which container each item is most likely to be
    stored. If no container is suitable, say so. Provide reasoning for
    each item.

Format:
Item: [Name]
Best container: [just container id or "None"]
Reasoning: [Your explanation]
"""

```

2. Story-Based Prompt We next tried a narrative approach, asking the model to complete a short story that ends with a storage decision:

```

prompt = (
    f"As they stepped into the kitchen, they began searching for a
      {item.lower()}, scanning the scene for storage areas - known as
      containers, like drawers, cabinets, or pantry doors where
      household items are typically kept.\n"
    "Based on the descriptions of the detected containers, they paused in
      front of one that seemed just right and reached toward...\n"
    "\n"
    "Finish the story by selecting the most suitable container from the
      list below, or say the item isn't in any container if none of
      them are appropriate:\n"
    f"{chr(10).join(f"- {desc}" for desc in containers)}"
)

```

Although this version was more natural and creative, it generally underperformed compared to the direct instructional format.

3. Structured Prompt with System and User Roles After learning more about prompt engineering, we adopted a structured format to balance informativeness and brevity. This prompt format allowed the model to internalize the task logic from the system message and focus on inference during deployment. It consists of two parts:

- **System Prompt:** Encodes general knowledge about the task, including comprehensive instructions and illustrative examples to prime the model.
- **User Prompt:** Provides a concise task instance, including the item to be located and the textual descriptions of all detected containers in the scene as presented above.

```

system_prompt = """
You are helping locate a household item in a kitchen.

```

The item is stored in one of several visible containers (e.g., drawers, cabinets), but I don't know which.

I'll provide a list of container descriptions and the item name.

Your task is to identify the most likely container based on typical kitchen organization. If none are suitable, return "None".

Response format:

Item: [Name]

Best container: [Container ID or "None"]

Reasoning: [Short explanation]

Example 1

Item: Fork

Containers:

- Container 1: cabinet door below the countertop, located to the right of the dishwasher.
- Container 2: below the countertop, located to the left of the dishwasher.
- Container 3: cabinet door above the countertop, located above the coffee machine.
- Container 4: drawer below the countertop.
- Container 5: cabinet door above the countertop.
- Container 6: cabinet door above the countertop.

Item: Fork

Best container: 4

Reasoning: Forks are usually stored in drawers below the countertop for easy access.

Example 2

Item: Trash Bag

Containers:

- Container 1: cabinet door above the countertop.
- Container 2: drawer below the countertop.
- Container 3: cabinet door below the countertop, located below the sink.

Item: Trash Bag

Best container: 3

Reasoning: Trash bags are commonly stored under the sink near the trash can.

Example 3

Item: Knife

Containers:

- Container 1: cabinet door.
- Container 2: drawer, located below the electronic kettle, above the oven, at the bottom-right of the refrigerator, and at the bottom-left of the dish drying rack.
- Container 3: cabinet door, located below the dishwasher.

```

Item: Knife
Best container: 2
Reasoning: Knives are typically stored in drawers for safety and
            accessibility.

### Example 4
Item: Baking pan
Containers:
- Container 1: drawer below the countertop, located at the bottom-right
  of the oven, and at the top-right of the electronic kettle.
- Container 2: cabinet door below the countertop, located to the right
  of the electronic kettle, and at the bottom-right of the oven.
- Container 3: cabinet door below the countertop, located below the
  stove.
- Container 4: below the countertop.
- Container 5: cabinet door below the countertop, located to the left of
  the electronic kettle, and at the bottom-left of the oven.

Item: Baking pan
Best container: 5
Reasoning: Baking pans are stored in cabinets below the countertop near
            the oven.

Now respond to the following:
"".strip()

user_prompt = f"Item: {item}\nContainers:\n" + "\n".join(f"- {desc}" for
  desc in containers)

```

References

1. Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A.C., Lo, W.Y., Dollár, P., Girshick, R.: Segment anything. In: 2023 IEEE/CVF International Conference on Computer Vision (ICCV). pp. 3992–4003 (2023). <https://doi.org/10.1109/ICCV51070.2023.00371>
2. Liu, S., Zeng, Z., Ren, T., Li, F., Zhang, H., Yang, J., Jiang, Q., Li, C., Yang, J., Su, H., Zhu, J., Zhang, L.: Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In: Computer Vision – ECCV 2024: 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part XLVII. p. 38–55. Springer-Verlag, Berlin, Heidelberg (2024). https://doi.org/10.1007/978-3-031-72970-6_3, https://doi.org/10.1007/978-3-031-72970-6_3
3. Peng, Z., Wang, W., Dong, L., Hao, Y., Huang, S., Ma, S., Wei, F.: Kosmos-2: Grounding multimodal large language models to the world. arXiv: 2306.14824 (2023), <https://arxiv.org/abs/2306.14824>
4. Qwen: Qwen2.5 technical report. arXiv: 2412.15115 (2025), <https://arxiv.org/abs/2412.15115>
5. Singh, A.: Meta llama 4: The future of multimodal ai. SSRN Electronic Journal p. 15 (04 2025). <https://doi.org/10.2139/ssrn.5208228>